



PROCESADORES DE LENGUAJE - trad

Grupo 206

11/03/2025

Jorge Mejías Donoso 100495807@alumnos.uc3m.es

Alberto Menchen Montero 100495692@alumnos.uc3m.es

Objetivo General del Proyecto

El propósito fundamental de este proyecto es la construcción de un traductor automático que transforme programas escritos en un subconjunto representativo del lenguaje C a una notación Lisp ejecutable en CLISP. Este proceso se apoya en la utilización de herramientas como Flex (para el análisis léxico) y Bison (para el análisis sintáctico y semántico), permitiendo así la creación de un compilador modular, extensible y alineado con las especificaciones formales de la práctica final.

El sistema ha de ser capaz de reconocer estructuras clave del lenguaje C, tales como funciones, bucles, expresiones aritméticas y lógicas, llamadas a funciones, y realizar su conversión a formas prefijadas en Lisp que respeten tanto la semántica como la estructura ejecutable del programa original.

Evolución desde la versión trad4.y hasta trad.y

1. Corrección crítica en el tratamiento de variables locales

Se ha resuelto un error determinante que provocaba la generación de nombres de variable incorrectos (como `null_variable`) en la primera función del programa. Este fallo se debía a la asignación tardía de la variable de control `funcion_actual`, cuyo valor se usaba para generar nombres únicos de variables locales en notación Lisp (por ejemplo, `main_x`, `fibonacci_n`). Ahora, la asignación se realiza en cuanto se detecta la cabecera de la función, garantizando la consistencia de los identificadores generados en todas las funciones, incluidas aquellas definidas en primer lugar.

2. Incorporación completa de estructuras del lenguaje C

La gramática ha sido extendida significativamente para soportar una gama más amplia de construcciones del lenguaje C. Entre las funcionalidades correctamente soportadas se encuentran:

- Declaración y uso de **variables globales y locales**, con nombre único prefijado por el identificador de la función.
- Definición de **funciones con y sin parámetros**, así como su invocación en cualquier contexto.
- Traducción de estructuras condicionales: `if`, `else`, y `return`.
- Implementación precisa de bucles `while`, que se traducen correctamente a `loop while` en Lisp.
- Soporte completo para **vectores** (arrays unidimensionales): declaración con `make-array`, lectura con `aref` y escritura con `setf (aref ...)`.

Esta expansión garantiza la cobertura de la mayoría de los programas típicos escritos en C básico.

3. Mapeo detallado de sentencias de salida (`printf`, `puts`)

Se ha refinado el tratamiento de instrucciones de salida para ajustarse con precisión al comportamiento de `printf` y `puts` en C:

- `puts("...")` se traduce directamente como `(print "...")`, asegurando un salto de línea.
- `printf(...)` con múltiples expresiones y cadenas se traduce en múltiples llamadas a `(princ ...)`, respetando el orden y el contenido de la cadena original.
- Se ha eliminado la aparición indeseada de valores de retorno de `princ` (por ejemplo, el número de caracteres impresos) mediante el uso de `progn` para agrupar y silenciar los efectos secundarios.

4. Proceso en curso: estructura del bucle `for`

Aunque se ha iniciado la implementación de la estructura `for`, se detectaron discrepancias respecto a la forma canónica que debe tomar en la salida Lisp. La dificultad principal radica en la separación y traducción correcta de las tres partes del `for` de C (inicialización; condición; actualización) dentro de una construcción `loop` Lisp válida.

Por tanto, se ha decidido posponer la finalización de esta funcionalidad hasta la próxima sesión de revisión con el profesor, en la que se confirmará la estructura deseada para los bucles de este tipo. Los ejemplos se traducen correctamente a Lisp, pero la traducción final con CLisp no es del todo correcta porque no se sigue exactamente la estructura. .

5. Superación de pruebas oficiales (`test.zip`)

Se ha verificado que el traductor `trad.y` supera correctamente **todos los casos de prueba proporcionados por el profesorado** en el paquete `test.zip`, **a excepción de los que contienen estructuras `for`**, que aún están en fase de ajuste.

Esto representa un gran avance respecto a la versión anterior, en la que aún existían errores de traducción, nombres de variables incorrectos, y generación incompleta del código Lisp.

6. Pendiente: resolución de conflictos sintácticos

A pesar del progreso funcional del traductor, actualmente el fichero `back4.y` presenta **conflictos de análisis LR**, tanto de tipo `shift/reduce` como `reduce/reduce`, que aún **no han sido abordados ni optimizados**. Estos conflictos están relacionados principalmente con ambigüedades en la definición de expresiones complejas, precedencias no especificadas entre operadores, y redundancias en la gramática.

Aunque por ahora no impiden el correcto funcionamiento del traductor en los casos de prueba proporcionados, es necesario abordarlos en próximas iteraciones para asegurar:

- La robustez de la gramática.
- La ausencia de comportamientos ambiguos o inesperados.

- Una menor penalización en la evaluación final del proyecto.

Se contempla realizar una limpieza estructurada y jerárquica de las producciones de expresiones, junto con la definición explícita de precedencias y asociatividades.

Conclusión y próximos pasos

La evolución del archivo `trad4.y` a `trad.y` representa un salto cualitativo en la madurez del traductor. El sistema actual es capaz de:

- Traducir funciones complejas y recursivas.
- Manejar correctamente vectores, condiciones y bucles `while`.
- Generar código Lisp ejecutable y limpio, compatible con CLISP.

El único aspecto pendiente es la implementación de la estructura `for`, que se encuentra planificada y avanzada, pero será finalizada tras la revisión correspondiente.

Este desarrollo garantiza que el proyecto cumple con los requisitos funcionales y sintácticos de la práctica final, presentando una base robusta y modular para la entrega definitiva del compilador.